Decision-Table Based Testing (DTBT)

Decision-Table Based Testing (DTBT) is a systematic software testing technique used to deal with situations where a software application has multiple conditions or rules that influence its behavior. This technique is especially useful when the program has complex decision-making logic with multiple conditions that can interact in different ways.

A **decision table** is a table that represents a set of conditions and their corresponding actions. It helps visualize all possible combinations of conditions (inputs) and the resulting actions (outputs). The goal of Decision-Table Based Testing is to ensure that every combination of conditions has been tested and the correct actions are taken in each scenario.

Key Components of a Decision Table:

1. **Conditions (Inputs):** These are the boolean (True/False) or other types of conditions that influence the decision-making process. Conditions are the factors that determine which actions will be performed.
2. **Actions (Outputs):** These are the actions or results that the system should take based on the evaluation of the conditions.
3. **Rules:** Each row of the decision table represents a unique combination of conditions and their corresponding actions. Each rule shows which action should be taken based on the combination of the condition states.

Structure of a Decision Table:

A decision table consists of:

- **Conditions** listed on the left side, typically with multiple possible values (True or False).
- **Actions** listed on the right side, which represent the outcomes or results of the condition combinations.
- **Rules** in the middle, representing all possible combinations of conditions.

A simple decision table looks like this:

Rule No	Condition 1	Condition 2	Action 1	Action 2
1	T	T	A	B
2	T	F	C	D
3	F	T	E	F
4	F	F	G	H

In the above table:

- **T** represents **True**.
- **F** represents **False**.
- **A**, **B**, **C**, etc., represent the actions that the system should take based on the conditions.

Example of Decision-Table Based Testing:

Consider a simple system where a discount is given to a customer based on their **membership status** and **purchase amount**.

Conditions:

- **Condition 1 (C1):** Is the customer a "VIP" member? (True/False)
- **Condition 2 (C2):** Is the purchase amount greater than $500? (True/False)

Actions:

- **Action 1 (A1):** Grant a "20% discount."
- **Action 2 (A2):** Grant a "10% discount."
- **Action 3 (A3):** Grant a "No discount."

Decision Table:

Rule No	Condition 1 (VIP)	Condition 2 (Purchase > $500)	Action 1 (20% Discount)	Action 2 (10% Discount)	Action 3 (No Discount)
1	T	T	Yes	No	No
2	T	F	No	Yes	No
3	F	T	No	No	Yes
4	F	F	No	No	Yes

Explanation of the Table:

- **Rule 1**: If the customer is a **VIP** and the purchase amount is **greater than $500**, the action is a **20% discount**.
- **Rule 2**: If the customer is a **VIP** and the purchase amount is **less than or equal to $500**, the action is a **10% discount**.
- **Rule 3**: If the customer is **not a VIP** and the purchase amount is **greater than $500**, the action is **No discount**.
- **Rule 4**: If the customer is **not a VIP** and the purchase amount is **less than or equal to $500**, the action is **No discount**.

Test Cases Based on the Decision Table:

1. **Test Case 1:**
    - **Input:** VIP = True, Purchase Amount = $600
    - **Expected Outcome: 20% Discount** (Rule 1)
2. **Test Case 2:**
    - **Input:** VIP = True, Purchase Amount = $400
    - **Expected Outcome: 10% Discount** (Rule 2)
3. **Test Case 3:**
    - **Input:** VIP = False, Purchase Amount = $600
    - **Expected Outcome: No Discount** (Rule 3)

4. **Test Case 4:**
    - **Input:** VIP = False, Purchase Amount = $400
    - **Expected Outcome: No Discount** (Rule 4)

Benefits of Decision-Table Based Testing:

- **Clear Representation:** Decision tables provide a clear and structured way of representing complex business logic.
- **Comprehensive Coverage:** By testing all possible combinations of conditions, decision-table testing ensures that all relevant scenarios are covered.
- **Reduces Redundancy:** Decision tables help avoid redundant test cases by organizing conditions and actions in a systematic way.

Limitations of Decision-Table Based Testing:

- **Complexity with Many Conditions:** As the number of conditions increases, the number of rules grows exponentially, making the decision table harder to manage.
- **Can Become Large:** For systems with multiple conditions, the decision table can become large and difficult to interpret.

Scenario-Based Exam Questions on Decision-Table Based Testing:

1. Scenario-Based Question 1:

Consider a bank's loan approval system with the following conditions:

- **Condition 1 (C1):** Does the customer have a credit score above 700? (True/False)
- **Condition 2 (C2):** Is the customer's income above $50,000? (True/False)

Actions:

- **Action 1 (A1):** Approve loan
- **Action 2 (A2):** Deny loan

- **Action 3 (A3):** Request more documents

Question:
Create a decision table for the above scenario and write the test cases that would cover all possible outcomes.

Answer:

Decision Table:

Rule No	Condition 1 (Credit Score > 700)	Condition 2 (Income > $50,000)	Action 1 (Approve Loan)	Action 2 (Deny Loan)	Action 3 (Request More Docs)
1	T	T	Yes	No	No
2	T	F	No	No	Yes
3	F	T	No	Yes	No
4	F	F	No	Yes	No

Test Cases:

1. **Test Case 1:**
   - **Input:** Credit Score = 750, Income = $60,000
   - **Expected Outcome: Approve Loan** (Rule 1)
2. **Test Case 2:**
   - **Input:** Credit Score = 750, Income = $40,000
   - **Expected Outcome: Request More Documents** (Rule 2)
3. **Test Case 3:**
   - **Input:** Credit Score = 650, Income = $60,000
   - **Expected Outcome: Deny Loan** (Rule 3)
4. **Test Case 4:**
   - **Input:** Credit Score = 650, Income = $40,000

- **Expected Outcome: Deny Loan** (Rule 4)

---

2. Scenario-Based Question 2:

A health insurance company provides the following conditions:

- **Condition 1 (C1):** Is the applicant under 30 years old? (True/False)
- **Condition 2 (C2):** Does the applicant smoke? (True/False)
- **Condition 3 (C3):** Does the applicant have a pre-existing condition? (True/False)

Actions:

- **Action 1 (A1):** Offer standard coverage
- **Action 2 (A2):** Offer premium coverage
- **Action 3 (A3):** Deny coverage

Question:
Create a decision table for the above scenario and write test cases to cover all possible actions.

Answer:

Decision Table:

Rule No	Condition 1 (Under 30)	Condition 2 (Smokes)	Condition 3 (Pre-existing Condition)	Action 1 (Standard Coverage)	Action 2 (Premium Coverage)	Action 3 (Deny Coverage)
1	T	T	T	No	Yes	No
2	T	T	F	Yes	No	No
3	T	F	T	Yes	No	No
4	T	F	F	Yes	No	No
5	F	T	T	No	Yes	No

Rule No	Condition 1 (Under 30)	Condition 2 (Smokes)	Condition 3 (Pre-existing Condition)	Action 1 (Standard Coverage)	Action 2 (Premium Coverage)	Action 3 (Deny Coverage)
6	F	T	F	No	Yes	No
7	F	F	T	No	No	Yes
8	F	F	F	No	No	Yes

Test Cases:

1. **Test Case 1:**
   - **Input:** Age = 25, Smokes = True, Pre-existing Condition = True
   - **Expected Outcome: Premium Coverage** (Rule 1)
2. **Test Case 2:**
   - **Input:** Age = 25, Smokes = True, Pre-existing Condition = False
   - **Expected Outcome: Standard Coverage** (Rule 2)
3. **Test Case 3:**
   - **Input:** Age = 35, Smokes = False, Pre-existing Condition = True
   - **Expected Outcome: Deny Coverage** (Rule 7)

Decision-Table Based Testing is a powerful technique for handling complex decision logic in software, where multiple conditions interact to produce different actions. By organizing all possible combinations of conditions and their corresponding actions in a decision table, testers can systematically create test cases that ensure complete coverage of all logical possibilities.